

PerX — Database Environment Setup Guide

Everything below covers PostgreSQL 16 + Redis 7 exactly as specified in CLAUDE.md and ADR-006. Follow it top to bottom — each section is a prerequisite for the next.

Prerequisites

Make sure these are installed on your machine before anything else:

- **Docker Desktop** (Mac/Windows) or **Docker Engine + Docker Compose plugin** (Linux) — v24+
- **Git** (to clone the repo)
- **Python 3.11+** (for running Alembic migrations)
- A terminal that can run bash commands

Verify Docker is working:

```
docker --version          # should say 24.x or higher
docker compose version    # should say v2.x
```

Step 1 — Clone the Repo & Set Up the .env File

```
git clone <your-perx-repo-url>
cd perx
```

Create the environment file at the project root:

```
cp .env.example .env
```

If .env.example doesn't exist yet, create .env manually with these exact values:

```
# — PostgreSQL —————
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=perx
POSTGRES_USER=perx_user
POSTGRES_PASSWORD=perx_secret

# Full async DSN (used by SQLAlchemy + Alembic)
DATABASE_URL=postgresql+asyncpg://perx_user:perx_secret@localhost:5432/perx

# Sync DSN for Alembic only (asyncpg doesn't support sync ops)
ALEMBIC_DATABASE_URL=postgresql+psycpg2://perx_user:perx_secret@localhost:5432/perx

# — Redis —————
```

```

REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0
REDIS_URL=redis://localhost:6379/0

# — Auth —————
JWT_SECRET_KEY=change_this_to_a_long_random_string_before_deploying
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7

# — Recommendation Engine —————
RECOMMENDER_WARM_THRESHOLD=10

# — Ollama (local LLM) —————
OLLAMA_BASE_URL=[http://host.docker.internal:11434] (http://host.docker
.internal:11434)
OLLAMA_MODEL=gemma2:2b
OLLAMA_TIMEOUT_SECONDS=10

```

Never commit .env to git. Make sure .gitignore has .env in it.

Step 2 — Start PostgreSQL & Redis via Docker Compose

Inside infra/docker-compose.yml, the DB-only services look like this (you should already have this from Phase 1 of the Codex execution — if not, create it now):

```

version: "3.9"

services:
  postgres:
    image: postgres:16-alpine
    container_name: perx_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "${POSTGRES_PORT}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s

```

```

        retries: 5

redis:
  image: redis:7-alpine
  container_name: perx_redis
  restart: unless-stopped
  command: redis-server --appendonly yes
  ports:
    - "${REDIS_PORT}:6379"
  volumes:
    - redis_data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5

volumes:
  postgres_data:
  redis_data:

```

Bring it up (from the infra/ folder or wherever your compose file lives):

```
docker compose -f infra/docker-compose.yml up -d postgres redis
```

Verify both containers are healthy:

```
docker compose -f infra/docker-compose.yml ps
```

You should see both perx_postgres and perx_redis with status healthy. If either shows starting wait 10 seconds and re-run.

Step 3 — Verify the Connections Manually

Test Postgres:

```
docker exec -it perx_postgres psql -U perx_user -d perx -c "\conninfo"
```

Expected output: You are connected to database "perx" as user "perx_user"

Test Redis:

```
docker exec -it perx_redis redis-cli ping
```

Expected output: PONG

Step 4 — Install Python Dependencies & Set Up Alembic

From the backend/ folder:

```
cd backend
```

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Make sure these are in requirements.txt for the database layer:

```
sqlalchemy==2.0.*
asyncpg==0.29.* # async Postgres driver (runtime)
psycopg2-binary==2.9.* # sync driver (Alembic only)
alembic==1.13.*
redis==5.0.*
pydantic-settings==2.2.*
python-dotenv==1.0.*
passlib[bcrypt]==1.7.*
```

Initialize Alembic if it hasn't been set up yet:

```
alembic init alembic
```

Then update alembic/env.py to point at your models and use the sync DSN:

```
# alembic/env.py - key changes only
from app.config import settings
from app.models import Base    # import all models so Alembic sees them

config.set_main_option("sqlalchemy.url",
settings.ALEMBIC_DATABASE_URL)
target_metadata = Base.metadata
```

Step 5 — Run the Initial Migration

Generate the migration from 001_initial.sql (or from your SQLAlchemy models if they're already written):

```
alembic revision --autogenerate -m "initial_schema"
alembic upgrade head
```

Verify every table got created:

```
docker exec -it perx_postgres psql -U perx_user -d perx -c "\dt"
```

You should see all tables from 001_initial.sql:

```
users
employee_profiles
employer_organizations
provider_profiles
perks
perk_selections
perk_interactions
budget_allocations
employee_wishlists
```

```
optimizer_runs
provider_ratings
notifications
refresh_tokens
```

If any are missing, check that your SQLAlchemy models import are included in alembic/env.py.

Step 6 — Verify the Redis Key Schema

Per ADR-006, the Redis key schema your app will use looks like this. Verify Redis is accepting writes with a quick manual test:

```
docker exec -it perx_redis redis-cli
```

```
# Inside the redis-cli shell:
```

```
SET budget:test_employer:test_employee:2025-06 10000
```

```
GET budget:test_employer:test_employee:2025-06
```

```
# → "10000"
```

```
SET session:test_jti "{\"user_id\": \"abc\", \"role\": \"employee\"}"
```

```
GET session:test_jti
```

```
# → the JSON you just set
```

```
DEL budget:test_employer:test_employee:2025-06
```

```
DEL session:test_jti
```

```
EXIT
```

If all three SET/GET operations work, Redis is healthy and the key schema is confirmed.

Step 7 — Seed the Database (for Development)

Before running the app for the first time you'll want non-empty tables so the recommender and dashboards have something to show. Create a seed script at backend/scripts/seed.py:

```
cd backend
```

```
python scripts/seed.py
```

The seed script should create at minimum:

- 1 employer organization
- 3–5 employee accounts (mix of cold-start and warm — some with 0 interactions, some with 12+)
- 2–3 provider accounts
- 10–15 perks across different categories (fitness, wellness, travel, food, learning)
- Some pre-existing perk_interactions rows so the warm-mode recommender has data to work with
- Budget allocations for each employee (e.g., €500/month)

Step 8 — Tear Down & Reset (when you need a clean

slate)

```
# Stop containers but keep volumes (data preserved)
docker compose -f infra/docker-compose.yml down

# Full reset — destroys all data, start fresh
docker compose -f infra/docker-compose.yml down -v

# Then bring it back up and re-run migrations
docker compose -f infra/docker-compose.yml up -d postgres redis
alembic upgrade head
python scripts/seed.py
```

Quick Reference — Common Commands

What	Command
Start DB + Redis	<code>docker compose -f infra/docker-compose.yml up -d postgres redis</code>
Check health	<code>docker compose -f infra/docker-compose.yml ps</code>
Postgres shell	<code>docker exec -it perx_postgres psql -U perx_user -d perx</code>
Redis shell	<code>docker exec -it perx_redis redis-cli</code>
Run migrations	<code>alembic upgrade head</code>
Rollback one step	<code>alembic downgrade -1</code>
Check migration history	<code>alembic history --verbose</code>
See current revision	<code>alembic current</code>
Seed data	<code>python scripts/seed.py</code>
Full reset	<code>docker compose -f infra/docker-compose.yml down -v</code>

Troubleshooting

psycopg2 install fails on Mac with Apple Silicon:

```
brew install libpq
pip install psycopg2-binary --no-cache-dir
```

asyncpg can't connect — "Connection refused": Make sure the Postgres container is fully healthy (not just running) before starting the FastAPI app. The healthcheck in Step 2 handles this — if you're starting the full stack at once, add `depends_on: postgres: condition: service_healthy` to your api service in docker-compose.

Alembic says "Target database is not up to date":

```
alembic stamp head # marks current DB state as up to date without
running migrations
alembic upgrade head # then run any new migrations on top
```

Redis PONG works but the app can't connect: Check REDIS_URL in .env — if the FastAPI app runs inside Docker, change localhost to the service name (redis). If it runs on the host, keep it as localhost.

Tables exist but data looks wrong after a model change:

```
alembic revision --autogenerate -m "describe_your_change"
alembic upgrade head
```

Never edit an already-applied migration file — always generate a new one.